

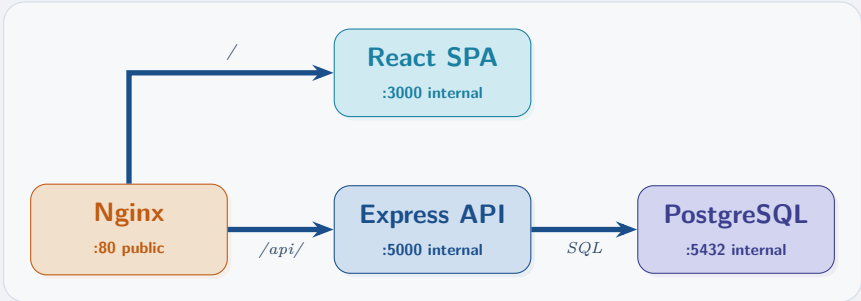
1.2pt2pt]

0.6pt1.5pt]

Project Architecture

& Automated Deployment Flow

A complete guide to the mono-repo application structure, CI/CD deployment pipeline, and everything from development through to production.



STACK

Node.js · React

PostgreSQL · Nginx

DEPLOYMENT

GitHub Actions

Docker Compose

INFRASTRUCTURE

OCI VM · OCIR

Docker · Docker Hub



Contents

1	Project Overview	2
1.1	Project Structure	2
2	Technology Stack	2
2.1	Back-end	3
2.2	Front-end	3
2.3	Database & DevOps	3
3	Service Architecture	3
3.1	High-Level Service Topology	4
3.2	Service Dependencies & Start Order	4
3.3	Service Details	5
4	Startup Sequence	5
5	Database Migration Strategy	6
5.1	End-to-End Migration Workflow	7
5.2	Migration File Format	7
5.3	Migration Tracking — pgmigrations Table	8
6	CI/CD Pipeline	8
6.1	Build Phase — GitHub Actions Runner	9
6.2	Deploy Phase — OCI Virtual Machine	10
6.3	Rollback Strategy	10
7	Environment Variables	10
7.1	Variable Reference	11
7.2	Secret Flow Diagram	11
7.3	.env File on OCI VM	11
8	Networking & Reverse Proxy	12
8.1	Network Architecture	12
8.2	Nginx DNS Resolution	12
8.3	End-to-End Request Flow	13
9	Deployment Process	13
9.1	Pre-Deployment Checklist	13
9.2	Triggering a Deployment	14
9.3	Verifying a Deployment on the OCI VM	14
10	Troubleshooting	14
10.1	Diagnostic Command Reference	15
11	Architecture Summary	15

1 Project Overview

This document describes a **production-grade, containerised full-stack application** consisting of four services orchestrated via a single `docker-compose.yml`. The mono-repo contains the backend, frontend, Nginx configuration, and the complete CI/CD pipeline definition.

Every `git push origin main` triggers a fully automated build, push, and deploy cycle. **Zero manual intervention is required** after the initial one-time VM setup.

1.1 Project Structure

Listing 1: Mono-repo directory layout

```
/root
  docker-compose.yml      # 4-service orchestration
  .env.example            # Environment variable template
  .gitignore

  backend/
    index.js              # Express server entry point
    Dockerfile
    config/db.js          # Connection pool
    migrations/           # node-pg-migrate files
    routes/user.routes.js

  frontend/
    src/                  # React source
    vite.config.js
    Dockerfile

  nginx/
    nginx.conf            # Reverse proxy rules
    Dockerfile

  .github/
    workflows/deploy.yml  # GitHub Actions CI/CD pipeline
```

2 Technology Stack

2.1 Back-end

Component	Version	Purpose
Node.js	22 Alpine	Runtime environment
Express.js	5.x	HTTP framework
pg	8.x	PostgreSQL driver
node-pg-migrate	8.x	Database schema versioning
CORS	2.8.x	Cross-origin resource sharing
dotenv	17.4.x	Environment variable loading
nodemon	3.1.x	Dev server auto-reload

2.2 Front-end

Component	Version	Purpose
React	18.x	UI component library
Vite	5.x	Build tool & development server
serve	14.x	Production static file server

2.3 Database & DevOps

Component	Version	Details
PostgreSQL	16 Alpine	Primary data store, persistent volume
Docker Compose	V2+	Multi-container orchestration
Nginx	Alpine	Reverse proxy, SSL termination ready
GitHub Actions	Native	CI/CD pipeline automation
OCIR	Native	Container image registry

3 Service Architecture

3.1 High-Level Service Topology

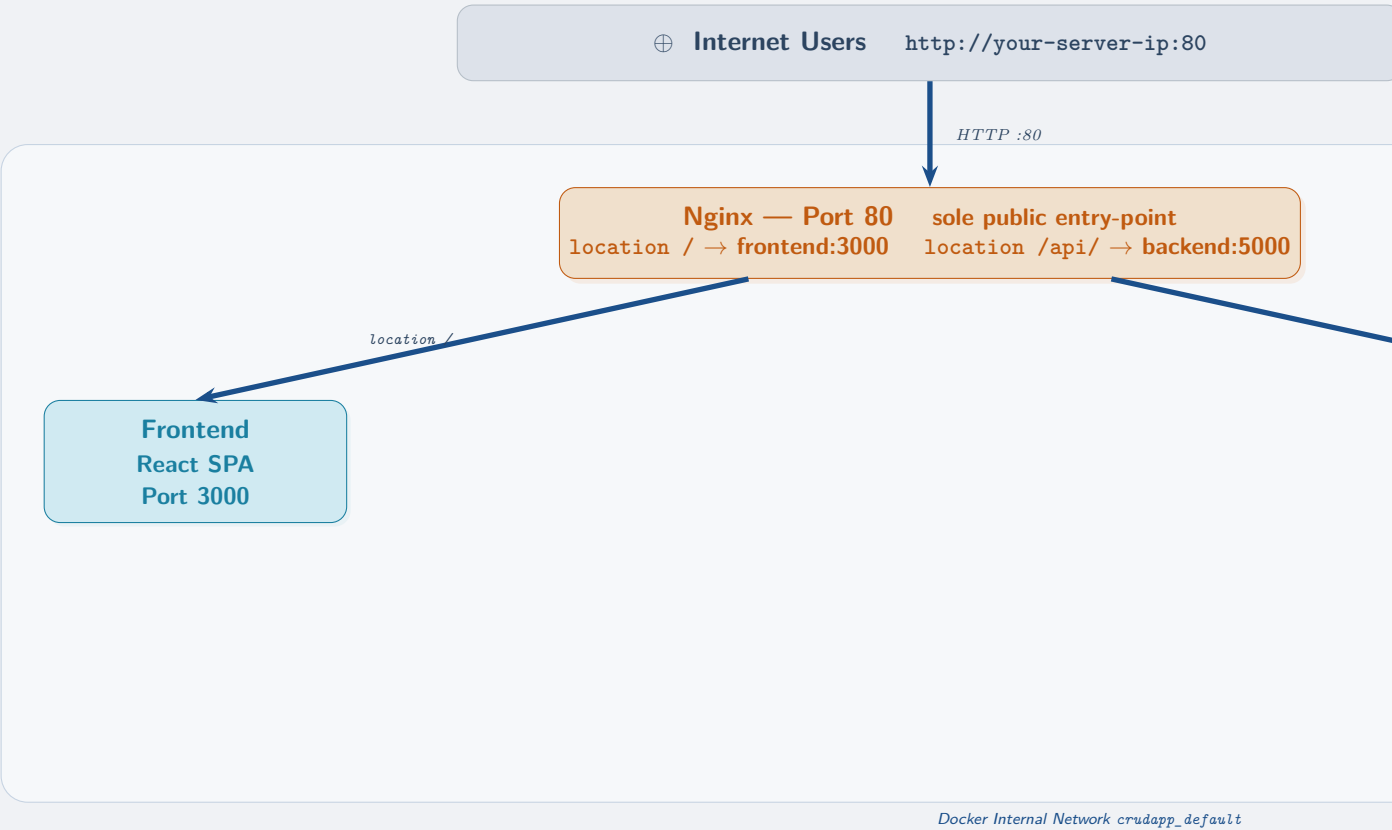


Figure 1: High-level service topology. Only Nginx exposes a public port. All inter-service communication uses Docker’s internal DNS.

3.2 Service Dependencies & Start Order

Docker Compose enforces a strict startup order via `depends_on` with `condition: service_healthy`.

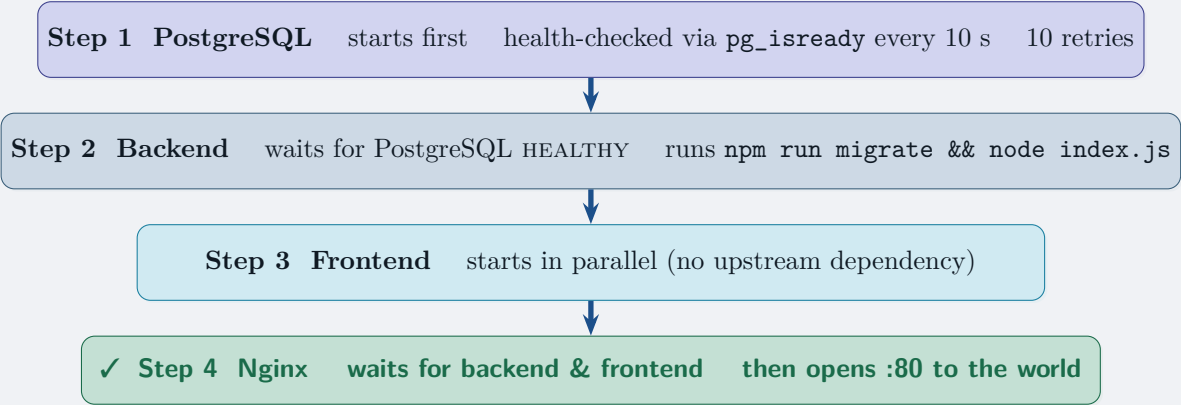


Figure 2: Enforced service start order and health-check dependency chain.

3.3 Service Details

3.3.1 PostgreSQL (Database)

- **Port:** 5432 — internal only, never exposed externally
- **Storage:** Persistent volume `pgdata:/var/lib/postgresql/data`
- **Memory tuning:** `shared_buffers=64MB, max_connections=50`
- **Health check:** `pg_isready -U $POSTGRES_USER` every 10 seconds

3.3.2 Backend (Express API)

- **Port:** 5000 — internal only
- **Startup command:** `npm run migrate && node index.js`
- **Responsibilities:** run DB migrations on every start, serve REST endpoints, authentication, connection-pool query execution

3.3.3 Frontend (React SPA)

- **Port:** 3000 — internal only
- **Runtime:** Node.js 22 Alpine, served by the `serve` package
- **Build:** Vite production build baked into the Docker image

3.3.4 Nginx (Reverse Proxy)

- **Port:** 80 — the *only* publicly exposed port
- **Routes:** `/` → `frontend:3000` `/api/` → `backend:5000` `/health` → `backend:5000`
- **DNS resolver:** `127.0.0.11:53` (Docker internal DNS) for dynamic hostname resolution

4 Startup Sequence

The complete chain of events triggered by `docker compose up -d`:

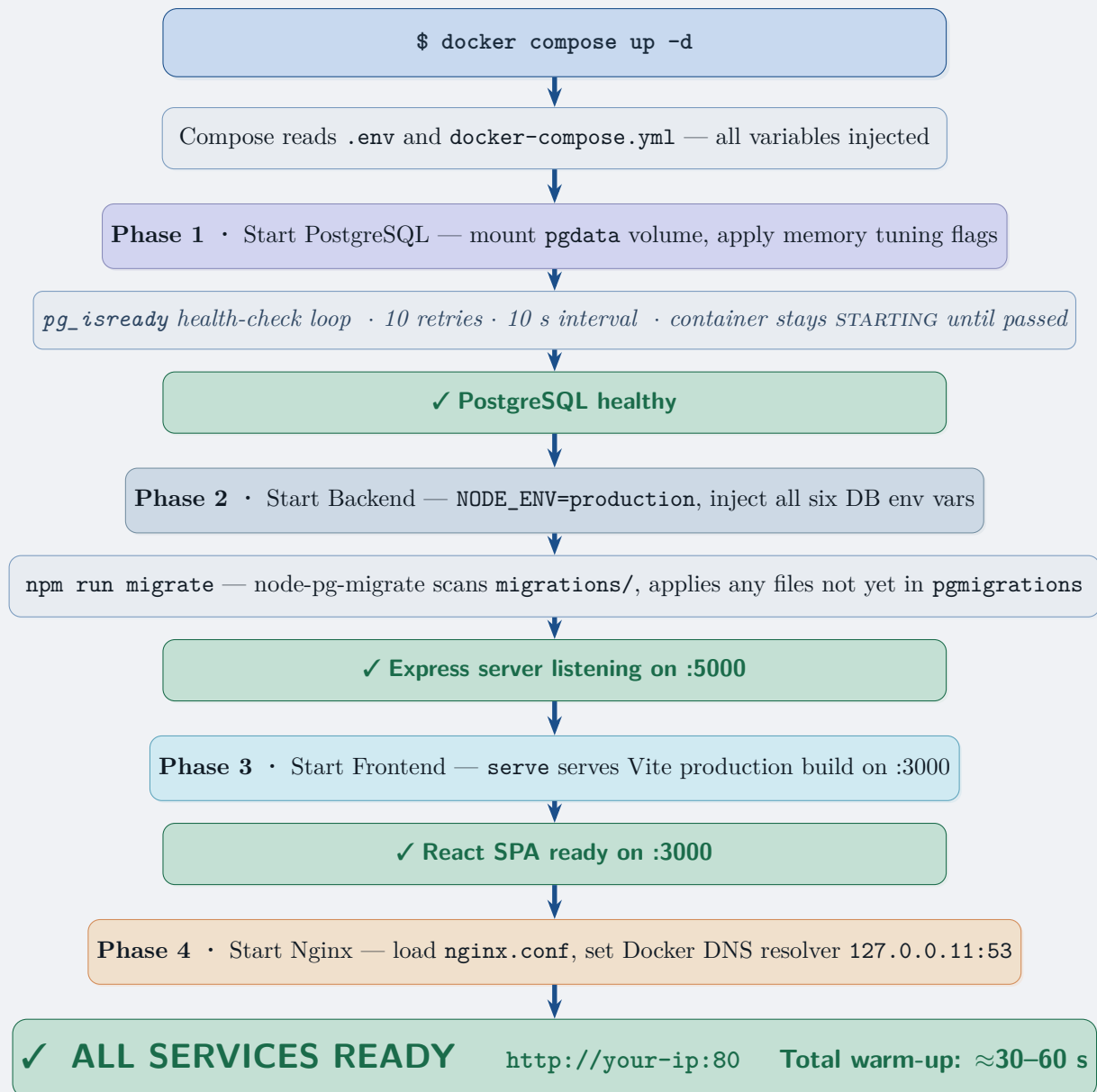


Figure 3: Complete startup sequence showing all four service phases and their health-check gates.

5 Database Migration Strategy

5.1 End-to-End Migration Workflow

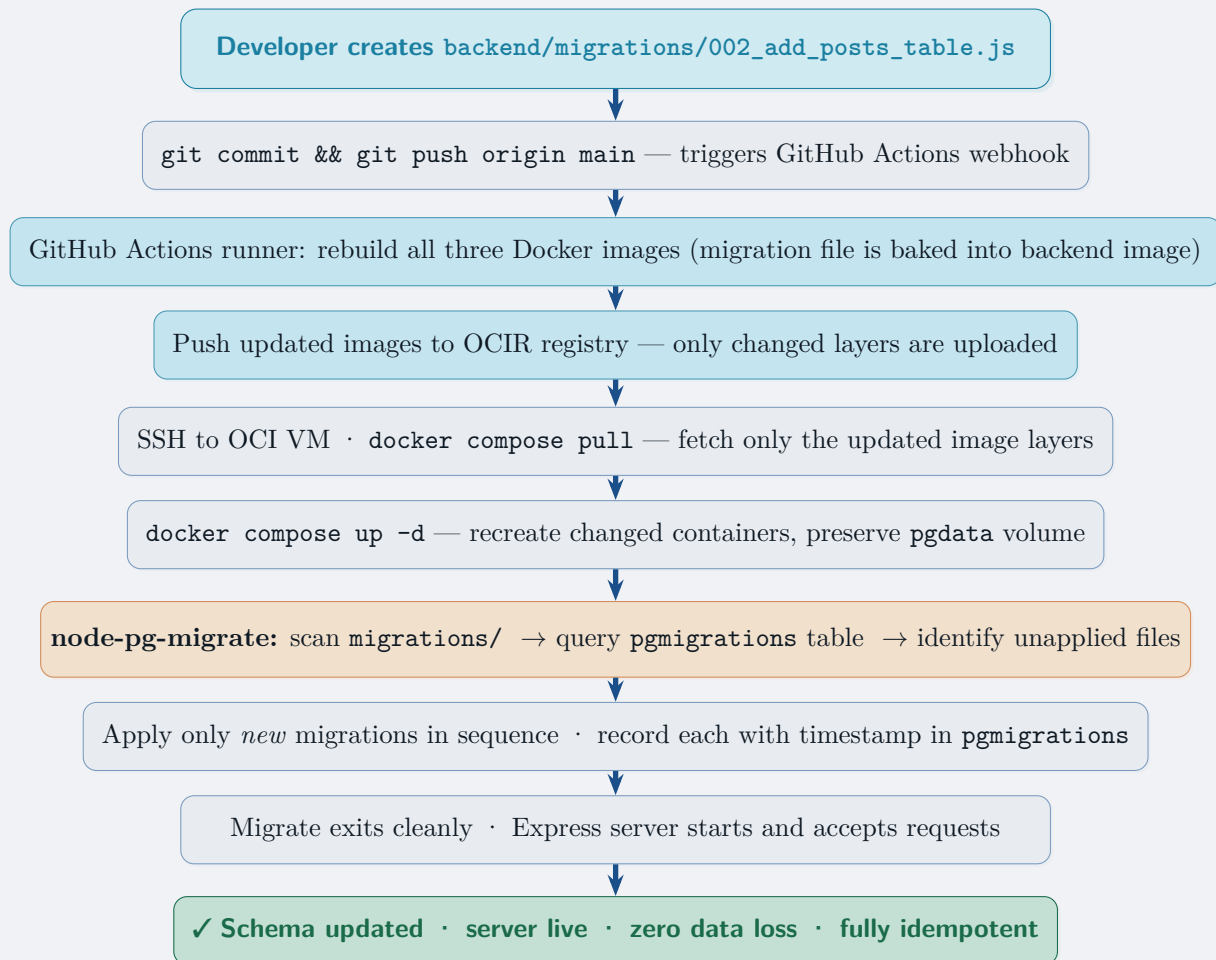


Figure 4: Full migration flow — from developer commit to live schema update.

5.2 Migration File Format

Listing 2: `backend/migrations/001_create_users.js`

```

exports.up = pgm => {
  pgm.createTable('users', {
    id:      { type: 'serial',      primaryKey: true },
    name:    { type: 'varchar(255)', notNull: true },
    email:   { type: 'varchar(255)', notNull: true, unique: true },
    password: { type: 'varchar(255)', notNull: true },
    created_at: { type: 'timestamp', default: pgm.func('NOW()') },
    updated_at: { type: 'timestamp', default: pgm.func('NOW()') },
  });
};

exports.down = pgm => pgm.dropTable('users');
```

5.3 Migration Tracking — `pgmigrations` Table

id	name	run_on
1	001_create_users	2025-05-01 10:30:15 UTC
2	002_add_posts_table	2025-05-02 14:22:08 UTC

Idempotency: Running `npm run migrate` twice never duplicates changes. `node-pg-migrate` skips any file already recorded in `pgmigrations`.

6 CI/CD Pipeline

The pipeline lives in `.github/workflows/deploy.yml` and fires on every push to the `main` branch. It has two phases: **Build** (on GitHub's hosted runner) and **Deploy** (on the OCI VM over SSH).

6.1 Build Phase — GitHub Actions Runner

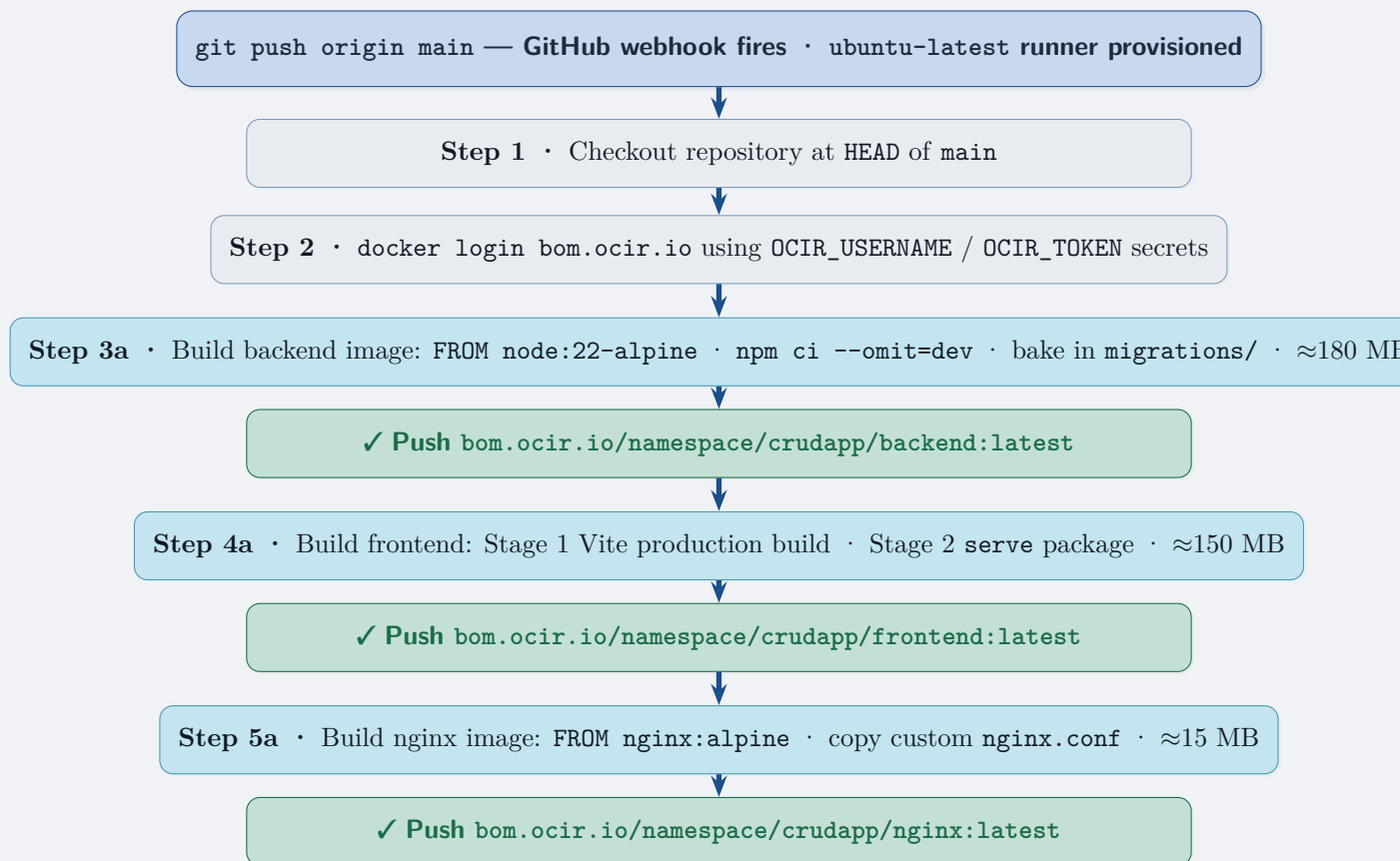


Figure 5: Build Phase: three images compiled on the GitHub runner and pushed to OCIR.

6.2 Deploy Phase — OCI Virtual Machine

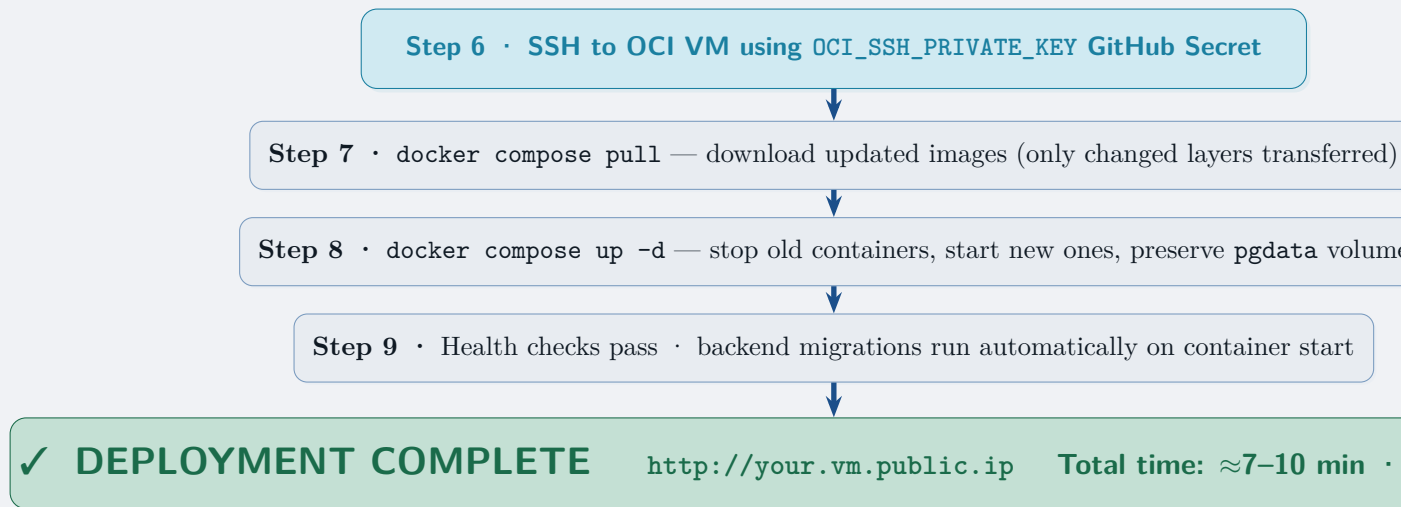


Figure 6: Deploy Phase: images pulled, containers restarted, migrations applied automatically.

6.3 Rollback Strategy

If a deployment breaks production, previous images remain in OCIR. SSH to the VM, edit `.env` to set `TAG=previous-working-tag`, then re-run `docker compose pull && docker compose up -d`. Alternatively, revert the commit on `main` and push again — the pipeline will rebuild and redeploy automatically.

7 Environment Variables

7.1 Variable Reference

Variable	Description
DB_NAME	PostgreSQL database name (<code>crudapp</code>)
DB_USER	PostgreSQL application user
DB_PASSWORD	Strong password — never commit to git
OCIR_REGISTRY	Registry endpoint (<code>bom.ocir.io</code>)
OCIR_REPO	Repository path (<code>namespace/crudapp</code>)
TAG	Image tag (<code>latest</code> or pinned version)
<i>GitHub Actions Secrets only — never stored in the <code>.env</code> file</i>	
OCIR_USERNAME	<code>namespace/user@email.com</code> for docker login
OCIR_TOKEN	OCIR authentication token
OCI_HOST	VM public IP address
OCI_USER	VM SSH username (<code>ubuntu</code>)
OCI_SSH_PRIVATE_KEY	PEM-format private key

7.2 Secret Flow Diagram

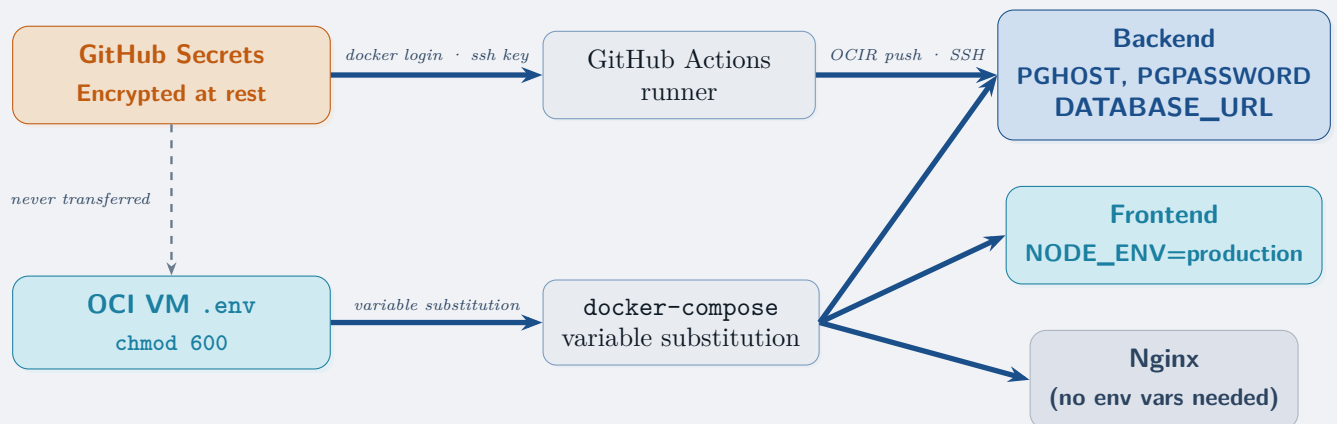


Figure 7: Secret flow: GitHub Secrets stay on the runner; only `.env` values reach containers on the VM.

7.3 `.env` File on OCI VM

Listing 3: Location: `/home/ubuntu/crudapp/.env` Permissions: 600

```

DB_NAME=crudapp
DB_USER=cruduser
DB_PASSWORD=YourStrong_P@ssw0rd!  # never commit this

```

```
OCIR_REGISTRY=bom.ocir.io
OCIR_REPO=your-namespace/crudapp
TAG=latest
```

8 Networking & Reverse Proxy

8.1 Network Architecture

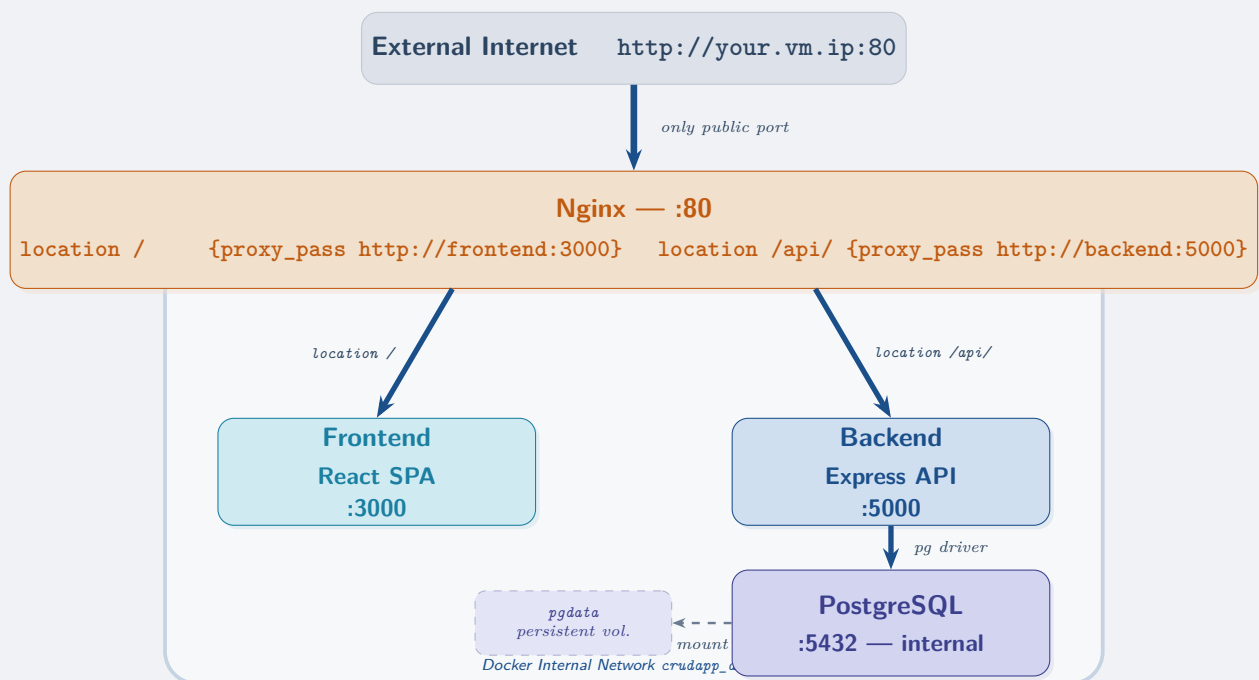


Figure 8: Complete network topology. PostgreSQL is never reachable from outside Docker's internal network.

8.2 Nginx DNS Resolution

Docker's internal DNS resolver at `127.0.0.11` allows Nginx to resolve container hostnames dynamically. Without this, Nginx would cache the IP at startup and fail if a container restarts with a new internal IP.

Listing 4: Relevant extract from `nginx/nginx.conf`

```
resolver 127.0.0.11 valid=10s;
resolver_timeout 5s;

location / {
    proxy_pass http://frontend:3000; # resolved via Docker DNS
}
```

```
location /api/ {
    proxy_pass http://backend:5000;
}
location /health {
    proxy_pass http://backend:5000/health;
}
```

8.3 End-to-End Request Flow

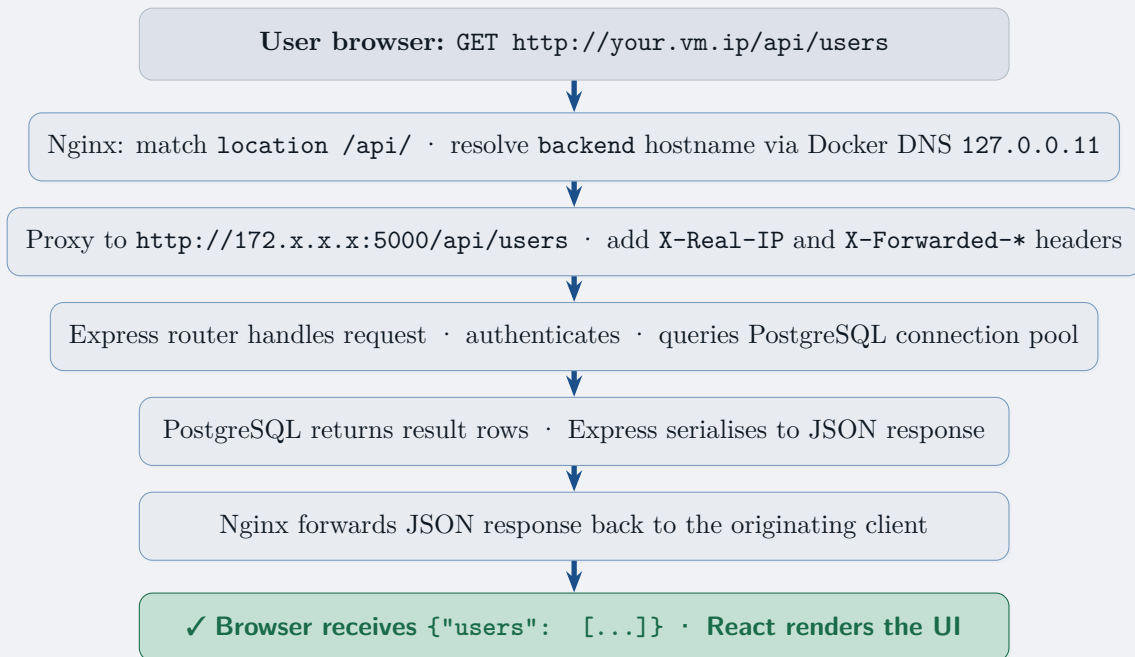


Figure 9: Complete request lifecycle for `GET /api/users`.

9 Deployment Process

9.1 Pre-Deployment Checklist

- ✓ All code committed and pushed to the `main` branch
- ✓ All 8 GitHub Actions Secrets configured (see Section 7)
- ✓ OCI VM `.env` file created with `chmod 600`
- ✓ OCI Security List: TCP Ingress rule added for port 80
- ✓ SSH private key registered on VM and matching GitHub Secret
- ✓ OCIR credentials verified in VM `.env`

9.2 Triggering a Deployment

Listing 5: Developer side — the only action ever needed to deploy

```
git add .
git commit -m "feat: add new feature"
git push origin main
# CI/CD pipeline starts automatically      no further action required
```

9.3 Verifying a Deployment on the OCI VM

Listing 6: Post-deployment verification commands

```
ssh ubuntu@your.vm.ip

# Confirm all 4 containers are running
docker ps

# Stream live logs per service
docker compose logs -f backend      # migrations + Express startup
docker compose logs -f frontend     # serve startup
docker compose logs -f nginx        # proxy requests
docker compose logs -f postgres     # health events

# Check which migrations have been applied
docker compose exec backend npm run migrate:status

# Quick health check
curl http://localhost/health
# Expected: {"success":true,"message":"Server is running",...}
```

10 Troubleshooting

Issue	Likely Cause	Resolution
Push image fails	OCIR credentials invalid	Verify Secrets: OCIR_USERNAME, OCIR_TOKEN, OCIR_REGISTRY
SSH connection fails	Key format wrong	Must be PEM format; run <code>ssh-keygen -p -N "" -m pem -f id_rsa</code>
App won't start on VM	Images not pulled	<code>docker compose pull --no-parallel</code> then <code>docker compose up -d</code>
Migrations fail	Syntax error in file	Check migrations/ files against node-pg-migrate docs
Backend can't reach DB	Wrong DATABASE_URL	Format must be: <code>postgres://user:pass@postgres:5432/db</code>
Frontend returns 404	Nginx config wrong	Ensure location / proxies to <code>http://frontend:3000</code>

Issue	Likely Cause	Resolution
Port 80 unreachable	OCI firewall	OCI Console → VCN → Security List → add TCP Ingress for port 80
Disk full on VM	Stale images	<code>docker image prune -a &&</code> <code>docker container prune</code>
Containers restarting	App crash on start	<code>docker compose logs {service}</code> to inspect the error
High CPU usage	Connection pool exhausted	Review <code>max_connections=50</code> in the postgres service config

10.1 Diagnostic Command Reference

Listing 7: Essential debugging commands on the OCI VM

```
# Stream all service logs
docker compose logs -f

# Inspect env vars inside a container
docker compose exec backend env | grep PG

# Query the migrations table directly
docker compose exec postgres \
  psql -U cruduser -d crudapp -c "SELECT * FROM pgmigrations;"

# Check disk and image usage
docker system df

# Restart a single service without a full redeploy
docker compose restart backend

# Network connectivity tests between containers
docker compose exec backend ping frontend
docker compose exec backend curl http://backend:5000/health
```

11 Architecture Summary

✓ Scalability	Each service is independently deployable and can scale horizontally without affecting others
✓ Reliability	Health checks, automatic restart policies, and persistent volumes prevent data loss
✓ Full Automation	Every <code>git push main</code> triggers build → push → deploy end-to-end with no manual steps
✓ Security	Secrets stay in GitHub; only <code>.env</code> lives on the VM with <code>chmod 600</code> ; no port other than 80 is externally reachable
✓ Maintainability	Clear service boundaries, versioned schema migrations, and structured container logs
✓ Production-Ready	Follows industry-standard containerised application architecture patterns

Every `git push main` triggers a full build-push-deploy cycle.
Database migrations run automatically on container start.
The `pgdata` volume preserves all data across every deployment.

Zero manual intervention required.
